

CARDIFF METROPOLITAN UNIVERSITY

ICBT Campus, Sri Lanka
BSc (Hons) Software Engineering

CIS6003 Advanced Programming

WRIT1: Individual Written Report

Sunrise Dental Clinic

Appointment and Patient Management System

Submitted by

W.A. Gangul Kalhara Rathnayake

Student ID: st20360306

Repository: <https://github.com/OGGY404/sunrise-dental-clinic>

September 2026

Table of Contents

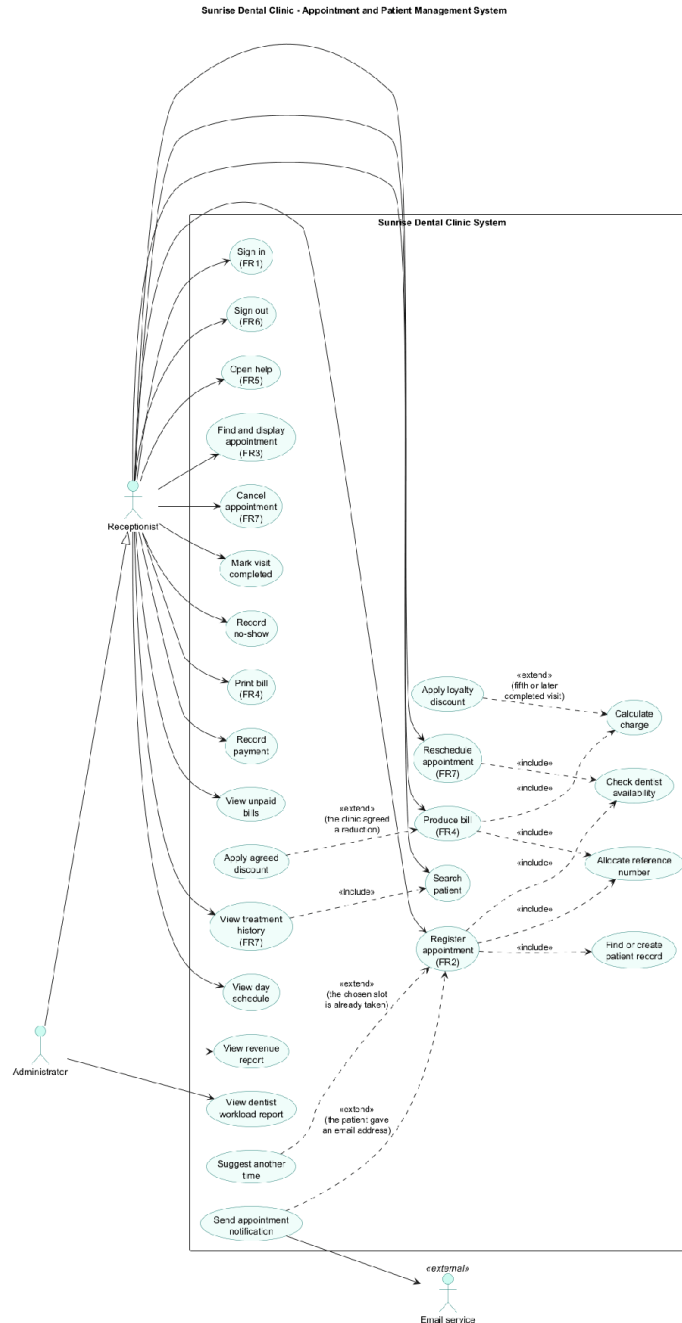
1. Introduction.....	2
2. Task A: System design with UML.....	3
2.1 Use case model.....	3
2.2 Class model.....	5
2.3 Sequence models.....	7
2.4 Documented assumptions.....	10
2.5 Critical evaluation of the design	10
3. Task B: Development.....	12
3.1 Three-tier architecture	12
3.2 Distributed application with web services	12
3.3 Design patterns.....	13
3.4 The database and its advanced features	14
3.5 Validation, sessions and security	16
3.6 The user interface	17
3.7 Additional functionality delivered	19
4. Task C: Testing	21
4.1 Rationale for the testing approach.....	21
4.2 How test-driven development was used.....	22
4.3 Test plan and test data	23
4.4 Test automation and results.....	24
4.5 Traceability matrix	25
4.6 Evaluation and lessons learned	26
5. Task D: Git and GitHub.....	27
5.1 Repository and branching strategy.....	27
5.2 Commits across multiple days	27
5.3 Continuous integration and deployment	28
6. Conclusion	30
References.....	31
Appendices.....	32

1. Introduction

The Sunrise Dental Clinic is a private dental clinic in Colombo where its schedules and files were recorded in writing. There are three repercussions mentioned in the scenario: double booking, missing files, and incorrect billing. These are simply three manifestations of one flaw, as the paper diary cannot ensure a rule.

This report covers the new computerized system replacement: the three-tier Java application implemented on Spring Boot 3 and MySQL 8, including the REST web services and seventeen Thyme leaf interfaces that satisfy six main functional requirements and thirteen additional ones listed in the brief. This application is thoroughly tested by 268 automated tests, eight UML diagrams and a pipeline for its building, testing and deployment. The four components to be evaluated come next; all code, diagrams and documentation can be found in the repository above.

2.1 Use case model



Assumption recorded on the diagram
1 Signing is assumed throughout. Every use case except "Sign in" requires the member of staff to be signed in already. Knowing that as an "individual" from all operations would fit the diagram in a mock, so it is stated once here. 2 Availability is checked twice, deliberately. The service <i>user</i> checks first, so the receiptpoint gets a sentence they can act on. The database checks last, with UNIQUE index, so two people serving in the same instant still cannot double book. 3 The administrator is a receiptpoint as well. The general <i>act</i> means every association / chain to Receiptpoint applies to administrator. 4 The email service is a supporting actor. The system talks to it. I never starts anything. Non-actors are switched out, real world credentials are supplied. 5 There are not actors. The system is the actor. The <i>use case</i> does not sign in. Giving them their own login is recorded as a real use case, not modelled here.

Figure 1: Use case diagram

Two humans act as actors within the system. The Receptionist is the one who carries out day to day tasks and the Administrator is depicted as a generalization of the Receptionist since an administrator does all the things that a receptionist does as well as viewing the management reports; the generalization ensures that all associations need to be depicted just once.

Email service acts as a supporting actor in that the system communicates with it but it never initiates anything. Dentists are intentionally not actors in the system since they view the day's printed schedule and do not log in. Providing dentists a login is documented as future work.

The stereotypes of <<include>> and <<extend>> are used in their appropriate forms, which are antonyms and are often confused.

- An include is an action that is performed by the base use case each and every time, extracted since more than one use case requires it. Appointment registration include Find or create patient record, since there cannot be an appointment without a patient. The arrow goes away from the base use case.
- An extension will take place sometimes depending on whether certain conditions are met; the arrow goes back to the base use case and is not aware that such an extension has taken place. Propose another extend of Register appointment when the chosen slot is taken. Four includes and four extends are modelled, each labelled with its condition.

2.2 Class model

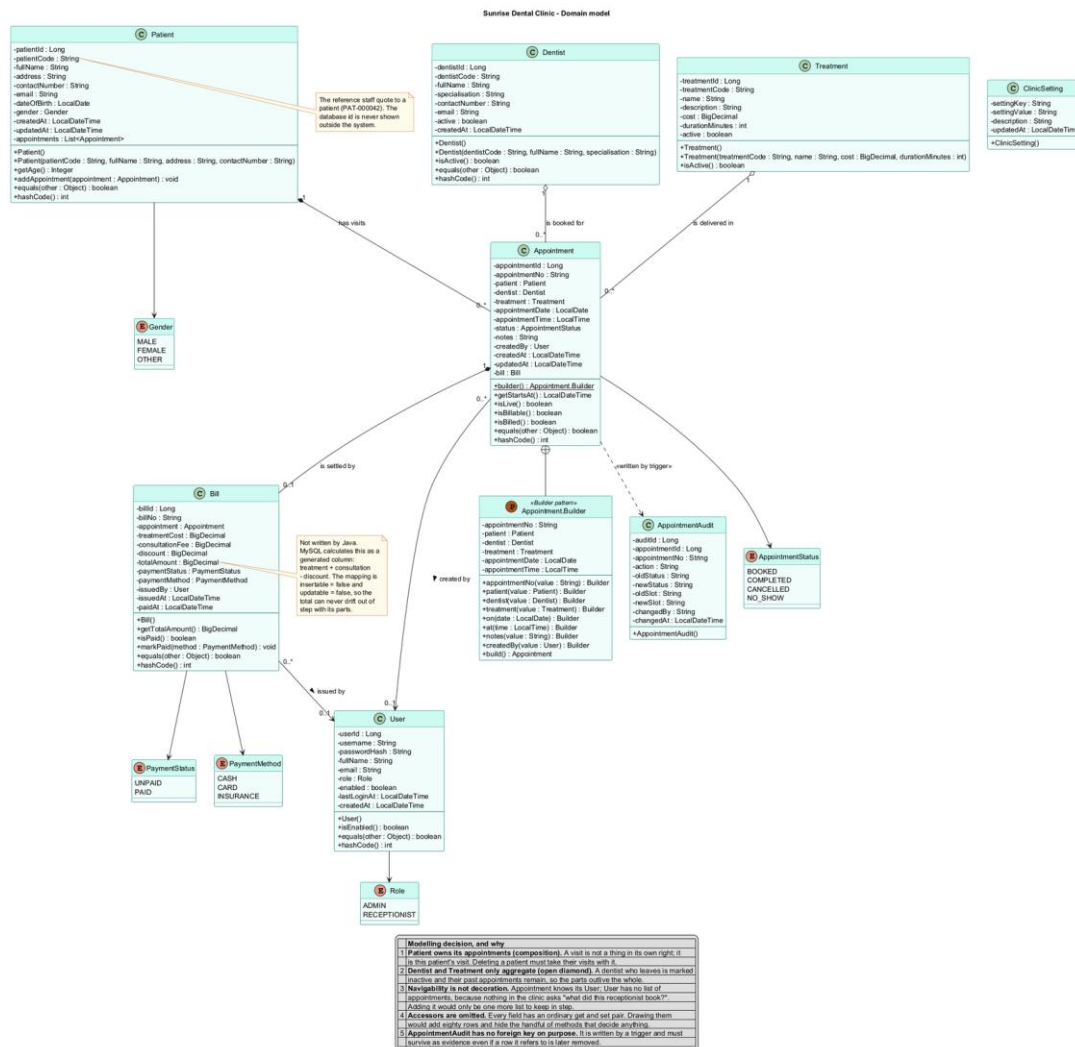


Figure 2: Domain class diagram

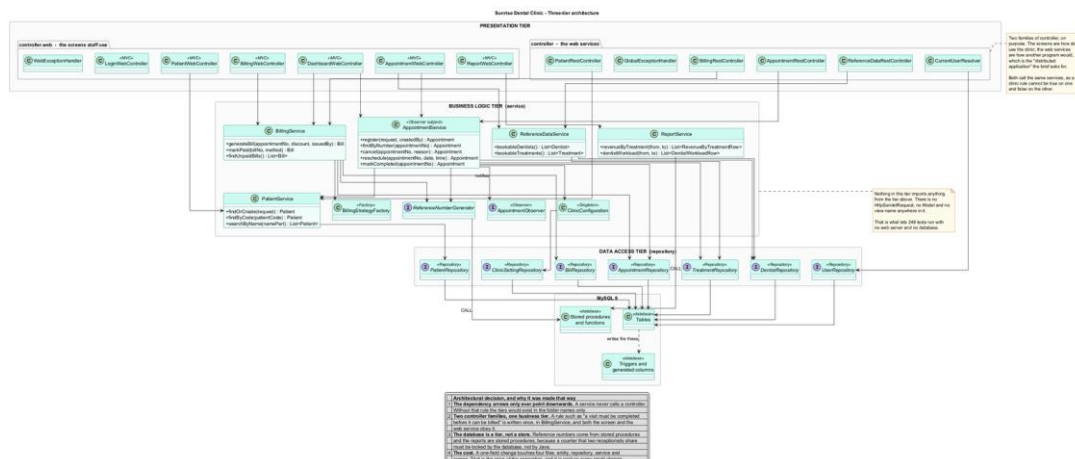


Figure 3: Three-tier class diagram

In Figure 2, each field has its type, and every method has its full signature, which has been copied from the source, not recalled. All the fields have been made private with the accessors as public.

Whole part relationships were the most challenging, and the two types have different uses for specific purposes:

- Composition (filled diamond) connects Patient to Appointment and Appointment to Bill. An appointment does not exist as an entity by itself; rather, it is a visit from this particular patient, and has no significance without the patient. The bill only exists as a payment for that particular visit.
- Aggregation (open diamond) connects Dentist and Treatment to Appointment. The distinction is that these components have a longer lifetime than the whole because an inactive dentist is flagged after they leave the clinic, but the appointments never get removed from the database.

Multiplicity is expressed on all associations. In Appointment 1 ↔ 0..1 Bill, the 0..1 multiplicity is the feature which allows a visit to occur but not to be billed yet.

Navigability depends on the code and not on beautifying the diagram. Appointment knows its User, but User does not have any collection of appointment since nothing in the clinic cares to ask, "What did this receptionist book?". Figure 3 organizes the same classes into tiers and the one message is that all arrows in dependencies point downwards.

2.3 Sequence models

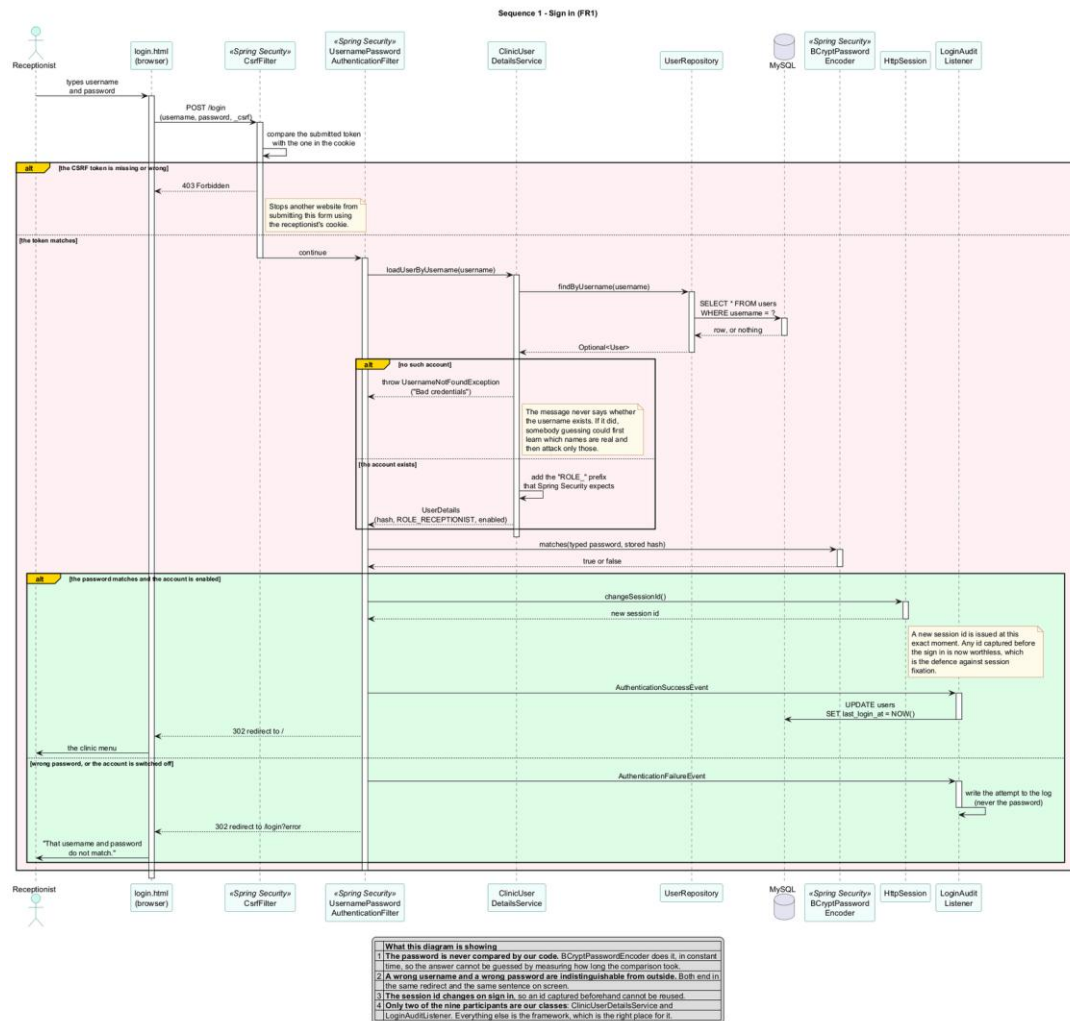


Figure 4: Sign in

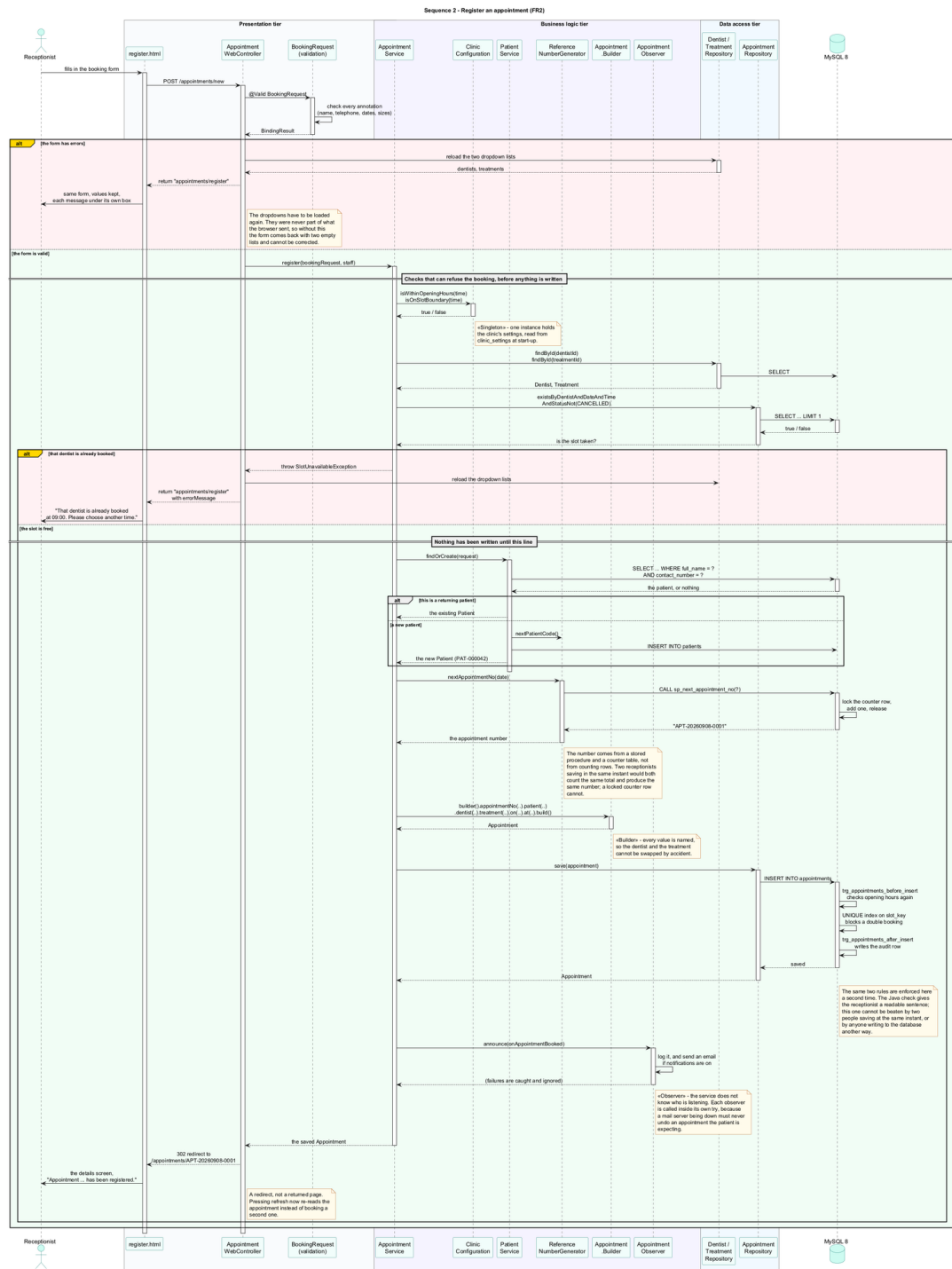


Figure 5: Register an appointment

the comment “Nothing has been written until this line” is indicative of the deliberate order of operations because all rejections come prior to creating the rows.

2.4 Documented assumptions

There were seventeen assumptions made during the modelling process; these can be found in Appendix D. The five most important to the design of the process include the assumption that the patient does not use the system personally; that one dentist will see one patient at a time, meaning that dentist date-time will be unique; that the returning patient will be identified on the basis of his/her name and telephone number because the name will be common between two individuals and the number common between the family members; that the visit will generate only one invoice and the prices will be copied onto it when it is issued to ensure the survival of the receipt beyond the increase in prices; and that cancellations free up the appointment whereas the no shows do not since the dentist's time has been wasted anyway.

There are two additional assumptions which are limitations rather than choices. The fact that a prolonged treatment will not yet preempt the next slot is true despite recording its own length, and some rules are applied twice, once in Java and once in SQL, as explained in §2.5.

2.5 Critical evaluation of the design

All three diagram types agreed on this, and I verified one such path myself. The use case of register appointment can be seen in Figure 1; its classes are shown in Figures 2 and 3 while the message flow is in Figure 5. They differed on the issue of navigability, where the class diagram I drew first showed User having a collection of appointments, but none of the sequence diagrams ever did this.

The most difficult choice was to apply some rules twice, once in Java code and again in SQL. I decided to keep both. Having one check in Java provides the secretary with a rule that can be followed, while having one check in the database prevents anyone from beating it, even if two users save at the same time. This is not free, since there are now two copies of a rule that might diverge, but I'd do it again.

The weakness in the design is in regard to the treatment time. The time for each treatment is tracked, but in case of a lengthy treatment, it doesn't yet lock out the next period of time. This was realized during the creation of the class diagram.

3. Task B: Development

3.1 Three-tier architecture

The code includes controller, service, and repository packages, along with model, dto and config packages, and one single rule applies to everything dependencies are only downward facing, which is the layering Fowler (2002) describes. There is nothing like `HttpServletRequest`, no Model and no view names in the business tier.

The proof here is not in package names but in the test coverage: out of 268 test cases, 262 run without a MySQL installation, which is made possible because the separation between the tiers is real. The 92 classes under `src/main` contain 6 services, 3 billing strategies, a factory, 4 notification and a configuration holder in the business tier.

3.2 Distributed application with web services

There are seventeen REST endpoints in the web service layer, built with the annotated controllers of the Spring framework (VMware, 2024a), making this a distributed application instead of a desktop monolith.

These include the creation, display, cancellation, rescheduling, and completion of a visit; patient search and treatment history; bill generation, reading, and settlement; and the two reference lists that the booking form requires. These are enumerated in Appendix I.

Two classes of controllers instead of one for the business layer have been deliberately used here: the regular controller returns JSON output for applications, and the `.web` controller returns HTML output for people. The two controllers use the same service layers, ensuring that a constraint like “A visit has to be completed before it is billed” does not become inconsistent across both interfaces.

3.3 Design patterns

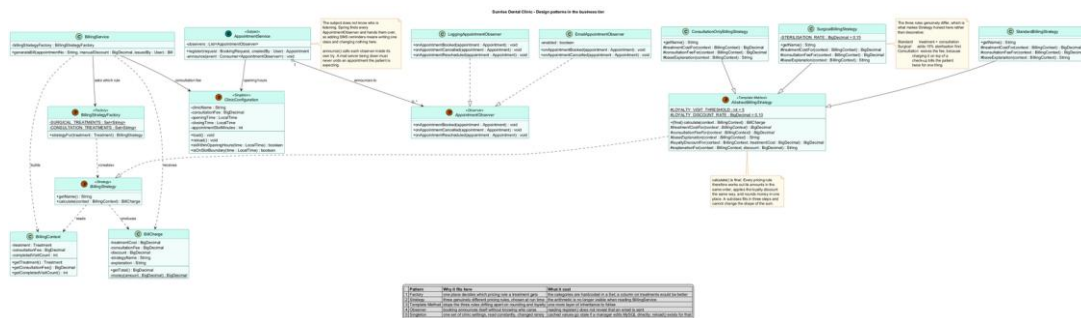


Figure 7: Design pattern class diagram

Eight patterns from the catalogue of Gamma et al. (1994) have been employed (Figure 7). Each one is identified through comments in the code, along with the improvement it brings and its cost since any pattern without a cost would normally not have been considered.

Pattern	Where	What it improved	What it cost
Singleton	ClinicConfigura tion	an example of how settings in the clinic are; read often and rarely changed	Values cached become out-of-date when a MySQL manager makes any changes to the database; this is where reload comes into play.
Strategy	three billing rules	pricings made during execution time when BillingService doesn't specify any rule	There is nothing in BillingService to suggest the arithmetic involved
Template Method	AbstractBilling Strategy	The method calculate() is final, which means that all three policies follow the same sequence and round off money only once.	one additional layer of genetics to trace
Factory	BillingStrategy Factory	One body makes decisions on the classification of the therapy	treatment types are hardcoded into a Set; it would be better to use a database field
Builder	Appointment.Bui lder	All values are labeled; therefore, dentistry and treatment cannot be confused by mistake.	about 40 additional lines, and another place to update
Observer	AppointmentObse rver,	Booking introduces itself without even	register() does not tell you that the email is

Pattern	Where	What it improved	What it cost
	ReminderNotifier	knowing the listener; SMS integration adds one extra class.	being sent
Repository	seven Spring Data interfaces	access to data is stated explicitly	The created SQL query does not appear in the code
MVC	controller.web + Thymeleaf	screens detached from regulations	two controller types to stay synchronized

Strategy is the pattern that does the most work in this system. All three of the pricing rules below really do differ:

- Standard: cost of the treatment + cost of the consultation.
- Surgical: includes 15 % sterilization surcharge to the cost of the treatment first (extraction, root canal, wisdom tooth).
- Consultation only: costs no consultation fee, as it would mean that the patient was charged twice for the same service (consultation).

This task could be accomplished in one pattern with an if, but all its branches would also have to include loyalty discount and rounding again.

3.4 The database and its advanced features

CIS6003 - database evidence

CIS6003 Advanced Programming - advanced database features in use

The 10 triggers created by triggers.sql:

TRIGGER_NAME	fires_on	on_table
trg_appointments_after_insert	INSERT	appointments
trg_appointments_after_update	UPDATE	appointments
trg_appointments_before_insert	INSERT	appointments
trg_appointments_before_update	UPDATE	appointments
trg_bills_before_insert	INSERT	bills
trg_bills_before_update	UPDATE	bills
trg_patients_before_insert	INSERT	patients
trg_patients_before_update	UPDATE	patients
trg_users_before_insert	INSERT	users
trg_users_before_update	UPDATE	users

Audit rows written by those triggers, and by no Java code:

appointment_no	action	old_status	new_status	changed_at
APT-20260915-0001	CREATED	NULL	BOOKED	2026-09-02 10:15:50
APT-20260915-0001	COMPLETED	BOOKED	COMPLETED	2026-09-02 10:15:51
APT-20260902-0001	CREATED	NULL	BOOKED	2026-09-02 11:08:39
APT-20260902-0002	CREATED	NULL	BOOKED	2026-09-02 11:08:39

total_amount is a GENERATED COLUMN - MySQL works it out, Java never writes it:

bill_no	treatment_cost	consultation_fee	discount	total_amount	payment_status
BIL-20260902-0001	28750.00	1500.00	500.00	29750.00	PAID
BIL-20260902-0002	6000.00	1500.00	500.00	7000.00	PAID
BIL-20260902-0003	2000.00	0.00	0.00	2000.00	PAID
BIL-20260902-0004	6325.00	1500.00	0.00	7825.00	UNPAID

Figure 8: Database evidence

The schema consists of 1,750 lines of SQL in four scripts, which include 10 tables, 6 stored functions, 13 stored procedures, and 10 triggers created automatically at the very first run. There are four cases when the code works in a way Java explicitly avoids.

Reference numbers are generated by stored procedures. Counting existing rows and adding one fails in the case of two receptionists updating at exactly the same time; each counts the same number of rows and generates APT-20260908-0007. `sp_next_appointment_no` locks one row in the counter for the amount of time it needs, so each receives 0007 and 0008, respectively.

Double booking is avoided by a generated column. `appointments.slot_key` contains `dentistId|date|time` for a live appointment and NULL for a cancelled appointment, with a UNIQUE index on it. Since MySQL allows multiple NULL values in a unique index (Oracle, 2024), cancelling makes the slot available but does not delete anything, and the visit will remain in the patient's history.

Total cost is computed in MySQL. `bills.total_amount` is a generated column that computes `treatment_cost + consultation_fee - discount`, with `insertable = false` and `updatable = false` so that Java can't update it. It cannot go out of sync with its parts, regardless of which program updated the row.

Triggers keep a record that no one can edit silently. Ten triggers do both validation and auditing (Figure 8). Two triggers write to `appointment_audit`, and this is a table never written to by any Java program; now the clinic can tell “who booked this appointment, and when?” an impossible question on paper.

Five stored procedures are invoked by the running system, not just distributed with it: two for reference numbers, and three for generating reports and alerts.

3.5 Validation, sessions and security

Sunrise Dental Clinic
Colombo

Home New appointment Find appointment Day schedule Patients Unpaid bills Help

reception RECEPTION Sign out

Register an appointment

Every visit gets its own appointment number. A patient who has been here before is recognised from their name and telephone number, so their treatment history stays in one place.

The patient

Full name
Kamal 123

Contact number — 0771234567 or +94771234567
077

A patient name can only contain letters, spaces, dots, hyphens and apostrophes. Please enter a valid Sri Lankan telephone number, for example 0771234567.

Address
No. 42, Galle Road

Email — optional
kamal@example.lk

Date of birth — optional
dd-yyy

Gender — optional
Not recorded

The visit

Dentist
Please choose a dentist

Treatment
Tooth Filling - Rs. 6,000.00

Please choose a dentist. The consultation fee is added to this when the bill is produced.

Date
dd-yyy

Time — the clinic is open 08:00 to 18:00, in half hours
08:00

Please choose a date for the appointment. Please choose a time for the appointment.

Notes — optional
For example: complains of pain in the lower left molar

Register the appointment Cancel

Sunrise Dental Clinic - Appointment and Patient Management System - CIS6003 Advanced Programming

Figure 9: Validation

Bean validation annotations protect every incoming field, and every restriction corresponds to the column length in schema.sql; therefore, anything that gets validated can be saved. A phone number will be valid only in the format 0771234567 or +94771234567, and nothing else; a name can consist of letters, spaces, full stops, hyphens and apostrophes, but not digits. The regular expression allows W.A.G.K. Rathnayake-Silva, which was erroneously denied by the previous letters-only format.

See Figure 9 for a denial; the exact same form comes back, along with a message for each problematic field and a reload of the two dropdown menus. The receptionist will have to type everything again otherwise.

Sessions and cookies. Login status is maintained via an HttpOnly session cookie, called SUNRISEID and SameSite=Lax, which means the browser does not attach the cookie when another website sends a request to ours, and this is what blocks crosssite form attacks. It expires in thirty minutes, which could be important if someone walks

away from his computer. A new session id is generated upon login, so an old one will be useless.

Cross-site request forgery protection is enabled, since saving login state in a cookie is what makes the attack possible (OWASP, 2024). The read operations require no tokens; all writes do.

Passwords are BCrypt hash values at work factor 10, compared by BCryptPasswordEncoder in constant time by Spring Security (VMware, 2024b), not any code written in this project. A bad username and a bad password will both be refused, so no user can figure out which usernames are registered. Role-based access control distinguishes between ADMIN and RECEPTIONIST: the reports show clinic-wide revenues, and the receptionist gets 403.

3.6 The user interface

The screenshot shows the main menu of the Sunrise Dental Clinic. The header includes the clinic name, location (Colombo), and navigation links: Home, New appointment, Find appointment, Day schedule, Patients, Unpaid bills, Help, reception, RECEPTION, and a Sign out button. The main content area features a greeting 'Good day, reception' for Wednesday, 2 September 2026. It displays three key metrics: 6 appointments booked for today, 1 bill not yet paid, and Rs. 7,825.00 still owed to the clinic. Below these are buttons for 'What would you like to do?' with options: Register an appointment, Find an appointment, Day schedule, Patients, Unpaid bills, and Help. The 'Today's diary (6)' section contains a table of appointments.

TIME	APPOINTMENT	PATIENT	DENTIST	TREATMENT	STATUS
09:00	APT-20260902-0001	Kamal Silva	Dr. Nimal Perera	Tooth Filling	COMPLETED
10:00	APT-20260902-0002	Nadeeka Fernando	Dr. Shanika Fernando	Braces Consultation	BOOKED
11:00	APT-20260902-0003	Kamal Silva	Dr. Nimal Perera	Dental Check-up	COMPLETED
14:00	APT-20260902-0004	Ruwan Jayawardena	Dr. Ruwan Jayasinghe	Tooth Extraction	COMPLETED
15:30	APT-20260902-0005	Anusha Perera	Dr. Anusha Wickrama	Child Dental Care	BOOKED
16:00	APT-20260902-0006	Saman Kumara	Dr. Nimal Perera	Scaling and Polishing	BOOKED

Sunrise Dental Clinic - Appointment and Patient Management System - CS6003 Advanced Programming

Figure 10: Main menu

Sunrise Dental Clinic

Colombo

[Home](#)
[New appointment](#)
[Find appointment](#)
[Day schedule](#)
[Patients](#)
[Unpaid bills](#)
[Help](#)

[reception](#)
[RECEPTION](#)
[Sign out](#)

APT-20260902-0002

BOOKED

registered on 2 Sept 2026, 11:08

The visit

DATE	TIME	DENTIST	TREATMENT
Wednesday, 2 September 2026	10:00	Dr. Shanika Fernando	Braces Consultation
TREATMENT COST			
Rs. 3,500.00			
Notes			
Wants advice about braces			

The patient

PATIENT NUMBER	NAME	CONTACT NUMBER	ADDRESS
PAT-000002	Nadeeka Fernando	0715558899	No. 7, Temple Road, Nugegoda

The patient has arrived

Mark the visit as completed

The patient did not arrive

Move this appointment

New date

02-Sep-2026

New time

10:00

Move the appointment

Cancel this appointment

The slot becomes free for another patient straight away. A reason has to be given, because it is written into the record.

Reason

For example: the patient telephoned to cancel

Cancel the appointment

Find another appointment

Back to the menu

Sunrise Dental Clinic - Appointment and Patient Management System - CIS6003 Advanced Programming

Figure 11: Appointment details

Sunrise Dental Clinic

Colombo, Sri Lanka

PATIENT BILL

BILL NUMBER	ISSUED
BIL-20260902-0002	2 Sept 2026, 11:08
PATIENT	APPOINTMENT
Kamal Silva	APT-20260902-0001

Tooth Filling	Rs. 6,000.00
Consultation fee	Rs. 1,500.00
Less discount	- Rs. 500.00
Total	Rs. 7,000.00

PAID

paid by CASH on 2 Sept 2026, 11:08

Thank you for visiting Sunrise Dental Clinic.

This bill was produced by the clinic system and needs no signature.

Figure 12: The printed bill

With 17 Thymeleaf pages using only one CSS file and only one navigation bar, all the jobs can be done with just one click as asked in the brief: menu driven approach. Home page (Figure 10) also presents the two numbers required by the front desk: total patients expected on that day and total amount due.

For data entry and result display we have two separate pages, as required by the assessment: registration page is appointments/register and the result page is appointments/view (Figure 11); billing page is bills/generate and bills/view. It is not a mere formality. Each save operation takes the user back to result page where pressing refresh reads the record again.

Figure 12 is a print layout, not the screen that has had the menu cut out; the print CSS leaves just the receipt, $6,000 + 1,500 - 500 = \text{Rs. } 7,000.00$.

Additionally, the system offers a guide to using the website (FR5) and a way to log out (FR6) via a POST request with CSRF token.

3.7 Additional functionality delivered

In addition to FR1- FR6, 13 more features were implemented which included: cancellation and re-scheduling, treatment history, double booking avoidance, role based user access, three types of reports, unpaid bills, and appointment reminders.

Sunrise Dental Clinic

Colombo

[Home](#)
[New appointment](#)
[Find appointment](#)
[Day schedule](#)
[Reminders](#)
[Patients](#)
[Unpaid bills](#)
[Help](#)

[reception](#)
[RECEPTION](#)
[Sign out](#)

Appointment reminders

Patients still booked for **Thursday, 3 September 2026**. The clinic also sends this round automatically each evening at six.

3 patients due in

2 can be emailed

1 need telephoning

Which day

Tomorrow

Show that day

Due in on 3 Sept 2026

TIME	APPOINTMENT	PATIENT	TELEPHONE	DENTIST	TREATMENT	HOW WE REACH THEM
09:00	APT-20260903-0001	Chamari Wijesinghe	0771230001	Dr. Nimal Perera	Tooth Filling	chamari@example.lk
10:30	APT-20260903-0002	Dinesh Rajapaksa	0771230002	Dr. Shanika Fernando	Scaling and Polishing	telephone them
14:00	APT-20260903-0003	Ishara Gunawardena	0771230003	Dr. Ruwan Jayasinghe	Tooth Extraction	ishara@example.lk

Send the reminders now

This does not wait for the evening job. Patients with an email address are emailed; the rest are listed above for you to telephone.

Send the reminders

Print this list

Back to the menu

Sunrise Dental Clinic - Appointment and Patient Management System - CS6003 Advanced Programming

Figure 13: Appointment reminders

Reminder function (figure 13) returns two numbers of patients instead of one: number of patients contacted by e-mail and number of patients without e-mail address. "Five reminders sent" seems to be quite an easy statement if there were two patients that could not be contacted anyway. This gives the indication of the receptionist whom one should call on the phone the thing that computer cannot do.

4. Task C: Testing

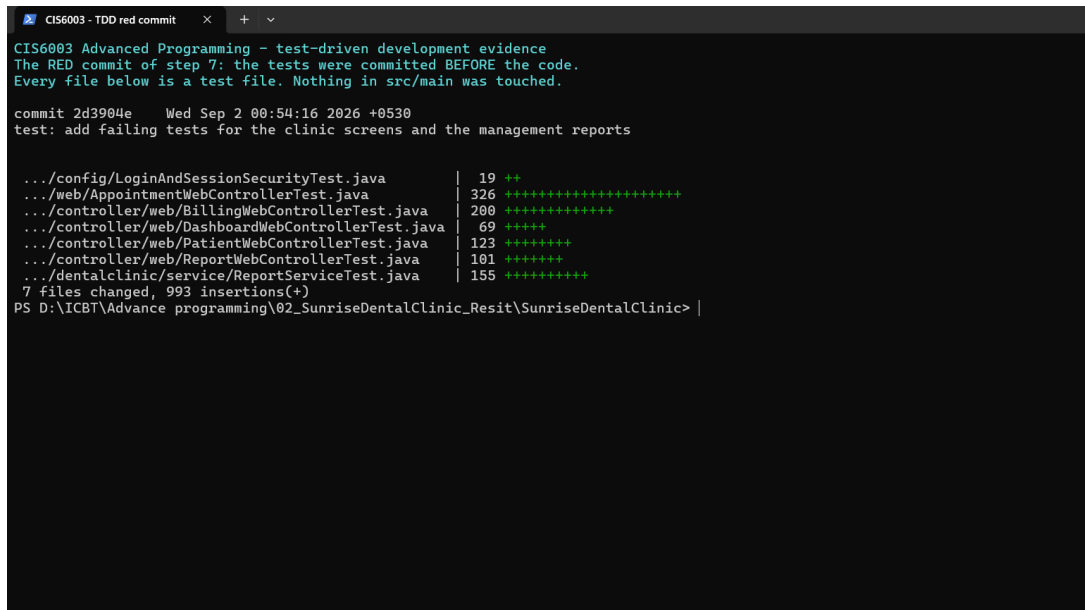
4.1 Rationale for the testing approach

Level	What it tests	Count	Runs against	Speed
Unit	one class, collaborators mocked	123	nothing	milliseconds
Repository	JPA mappings and derived queries	38	in-memory H2	~1 s
Web slice	one controller, services mocked	82	nothing	~1 s
Integration	the whole application context	19	H2, and a real port	~10 s
Database	schema, procedures, triggers	6	real MySQL 8	~10 s

I have mostly positioned the tests at their best locations. The rules of the clinic change the most frequently, hence, 123 unit tests can be found at the level which will detect any error in less than one minute and does not require any installation. This quickness made me agree to run the suite after every small change, and any suite which I do not want to run is worthless.

While six database tests may seem to be much fewer compared to 123, the number does not matter. This set of tests includes what cannot be detected by any other level, and all the tests are slow since they require starting up the real MySQL. Adding more tests will not help and will make the suite too slow, thus, making me stop running it.

4.2 How test-driven development was used



```
CIS6003 - TDD red commit x + v
CIS6003 Advanced Programming - test-driven development evidence
The RED commit of step 7: the tests were committed BEFORE the code.
Every file below is a test file. Nothing in src/main was touched.

commit 2d3904e Wed Sep 2 00:54:16 2026 +0530
test: add failing tests for the clinic screens and the management reports

.../config/LoginAndSessionSecurityTest.java | 19 ++
.../web/AppointmentWebControllerTest.java | 326 ++++++
.../controller/web/BillingWebControllerTest.java | 200 ++++++
.../controller/web/DashboardWebControllerTest.java | 69 +++++
.../controller/web/PatientWebControllerTest.java | 123 ++++++
.../controller/web/ReportWebControllerTest.java | 101 ++++++
.../dentalclinic/service/ReportServiceTest.java | 155 ++++++
7 files changed, 993 insertions(+)
PS D:\ICBT\Advance programming\02_SunriseDentalClinic_Resit\SunriseDentalClinic> |
```

Figure 14: The red commit

TDD is not made out to be the case here; it is evident from the Git logs. In seven different instances over the course of four days, the failed tests are checked into the repository before the code that passes the tests. In all the red commits, none of the tested classes actually exists.

Step	Red commit	Green commit	Tests after
Domain model	5c5c357	a05f7c4	39
Design patterns	2e89427	dab3032	77
Service layer	4fa3ebc	49c5559	121
Web services	ca70acc	1e07179	183
Security	a41b34b	2e068c6	205
The screens	2d3904e	ee29194	248
Reminders	22e09fc	7c8b027	262

All of them can be proved:

```
git show 2d3904e --stat      # only test files
git checkout 2d3904e
mvnw test-compile           # fails: cannot find symbol
AppointmentWebController
```

The refactoring, which Beck (2002) identifies as the most frequently skipped stage, is also documented. The three pricing algorithms were initially developed as three separate classes, and all the tests for them were successful. Upon reviewing them, one would see that each had those three common steps, and therefore `calculate()` was extracted to an abstract superclass and marked final. None of the tests needed changing. This was how the refactoring became safe, and this was the exact purpose of the tests.

Two things that the red step introduced that need mentioning are these. Writing `IsAllowedToday` before the annotation made one think "Is today past or future?" and write `@Past`, which was right because a patient who complains about his teeth can come to you in the same day. The reason for writing `FailingObserverDoesNotLoseTheBooking` before `IsObserverOnline` is that we need to call every observer in a try.

4.3 Test plan and test data

Appendix B contains all the test cases: 111 of them, with ID number, preconditions, test steps, test data, expected/actual results, pass/fail status and the automation for the test case. All test cases were automated; no manual test cases were created.

Test data were created according to the specification (column width, working time, rules stated in the brief) and where necessary created in order to investigate the risks, using equivalence partitioning and boundary value analysis.

See Appendix B for some instances: 101 letters in a name versus the 100-letter column, one letter less in a telephone number, yesterday versus today, and the fourth visit versus the fifth visit based on the loyalty rule.

The most significant example is TC-DB-05, where the duplicate field gets inserted directly into the table through `JdbcTemplate` without any checks performed by Java. This is the sole way to confirm that a rule exists in the database, not just in the application.

4.4 Test automation and results

```
D:\ICBT\Advance programmi x + v
CIS6003 Advanced Programming - Sunrise Dental Clinic
Running the full automated test suite: .\mvnw.cmd test

2026-09-02T22:42:35.024+05:30 INFO 16380 --- [Sunrise Dental Clinic] [main] l.i.c.d.service.ReminderService
s) ahead: There are no appointments booked for that day, so no reminders were needed.
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.203 s -- in lk.icbt.cis6003.dentalclinic.service.Remin
rs
[INFO] Running lk.icbt.cis6003.dentalclinic.service.ReminderServiceTest$ReadingTheList
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in lk.icbt.cis6003.dentalclinic.service.Remin
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.208 s -- in lk.icbt.cis6003.dentalclinic.service.Remin
[INFO] Running lk.icbt.cis6003.dentalclinic.service.ReportServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.030 s -- in lk.icbt.cis6003.dentalclinic.service.Repor
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 262, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ dental-clinic ---
[INFO] Loading execution data file D:\ICBT\Advance programming\02_SunriseDentalClinic_Resit\SunriseDentalClinic\target\jacoco.e
[INFO] Analyzed bundle 'Sunrise Dental Clinic' with 75 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 28.709 s
[INFO] Finished at: 2026-09-02T22:42:35+05:30
[INFO]
PS D:\ICBT\Advance programming\02_SunriseDentalClinic_Resit\SunriseDentalClinic> |
```

Figure 15: The suite passing

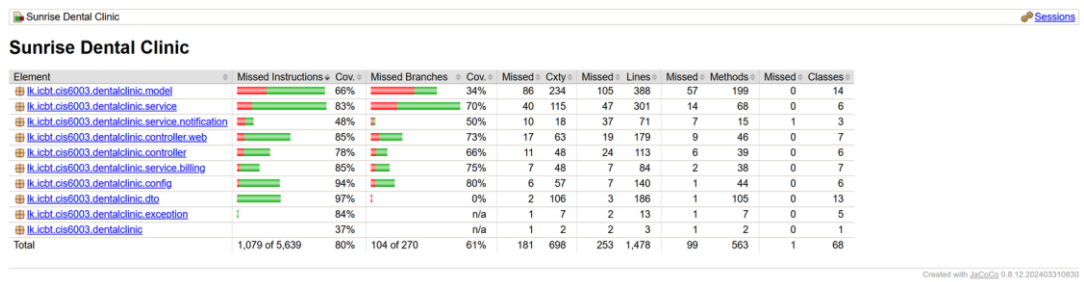


Figure 16: Coverage

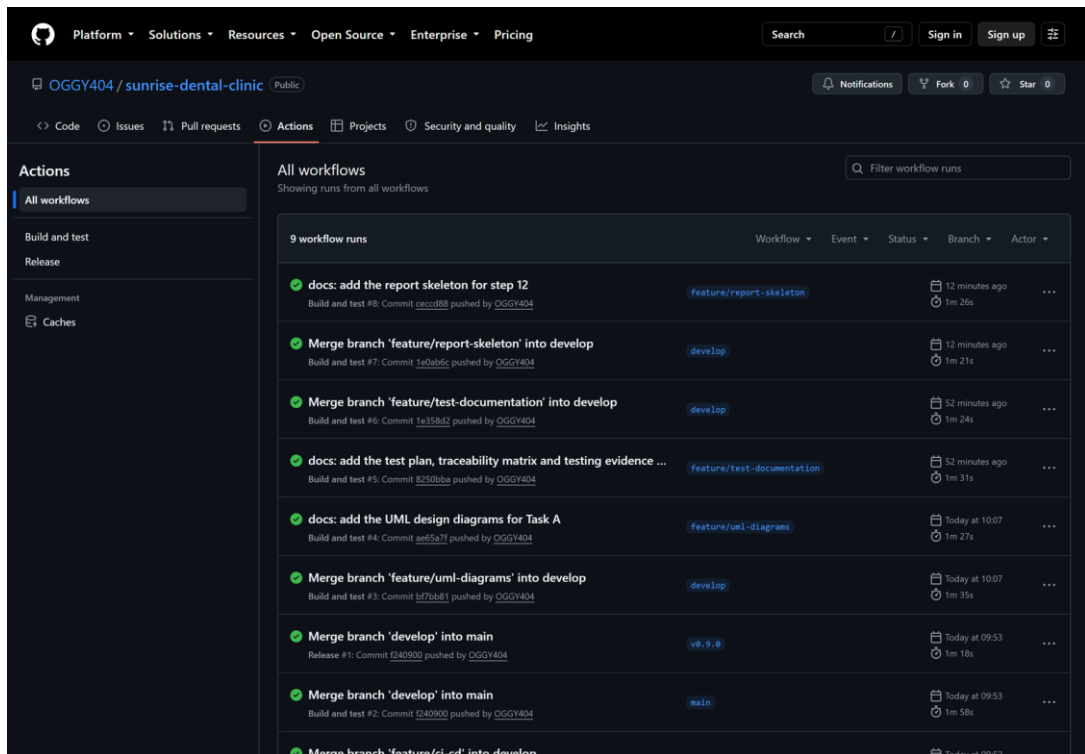


Figure 17: Continuous integration

268 automated tests in 32 test classes without a single failure (Figure 15). Coverage is 78.8 % instructions and 62.2 % branches according to JaCoCo (Figure 16). All this happens through a single command that requires no human intervention at all: ci.yml (Figure 17) executes the entire test suite on every push, reruns the database part with a real MySQL 8 container and builds the jar file if both tests succeeded.

Two numbers have been deliberately kept low. model is at 66.7% because most of it consists of accessors which are not invoked by any test, but those methods which make decisions are tested; service.notification is at 32.6% because emails are disabled till valid credentials are provided, and an increase in coverage will require mocking a mail server to test Spring's ability to send emails.

4.5 Traceability matrix

Appendix C shows all the requirements from the design it is based on, to the implementation in the code, and from there to the testing process. If a requirement has no corresponding entry in the testing column, it has not been tested at all, but there is none like that. The six basic requirements are listed below:

FR1 requires nine tests, FR2 thirty, FR3 six, FR4 twenty, FR5 one, FR6 two, and all thirteen features of FR7 need forty-four tests.

There are three requirements that cannot be automated according to the matrix, which states that: printing on paper, sending an actual email, and the look of the interface. A matrix showing total coverage is misleading.

4.6 Evaluation and lessons learned

The test suite missed some items, and this is the most important thing that I have learned. Three bugs have been found in the working program with all tests passed.

Bug	Why every test passed
password_hash had been CHAR(60) in SQL but VARCHAR in the entity, thus the application could not start on MySQL	Tables are built by H2 from entities; therefore, they can never differ
The CSRF cookie was not issued again after deletion on logging in, hence all saves done after logging in would return 403	MockMvc does not read the cookie but creates its own token
The two had been using different collations, so it was not possible to compare the parameter from the stored function to the column value. Every booking failed.	H2 has no stored functions at all

The problem in all three cases is the same: H2 and MockMvc were providing one side of the interaction and verifying the other side, hence the suite was validating its own assumptions rather than validating them.

I introduced two new layers in consequence: six tests using a real MySQL database, and two that would drive a real HTTP server with a real cookie, and I verified both of them by removing the fix and seeing them fail, which is the only way to make sure that a regression test works. The next time, I would have started my database tests in step 2, where the schema was created, rather than in step 9; an hour would have been enough to save all three bugs.

In retrospect, the practice of TDD was definitely a worthwhile one, though not necessarily for the reasons that I had imagined. TDD found less bugs than I had initially thought, but rather required me to think more carefully about how certain decisions were made, like the fact that `todayIsAllowed` had to be written before the annotation.

5. Task D: Git and GitHub

5.1 Repository and branching strategy

The public repository is <https://github.com/OGGY404/sunrise-dental-clinic>.

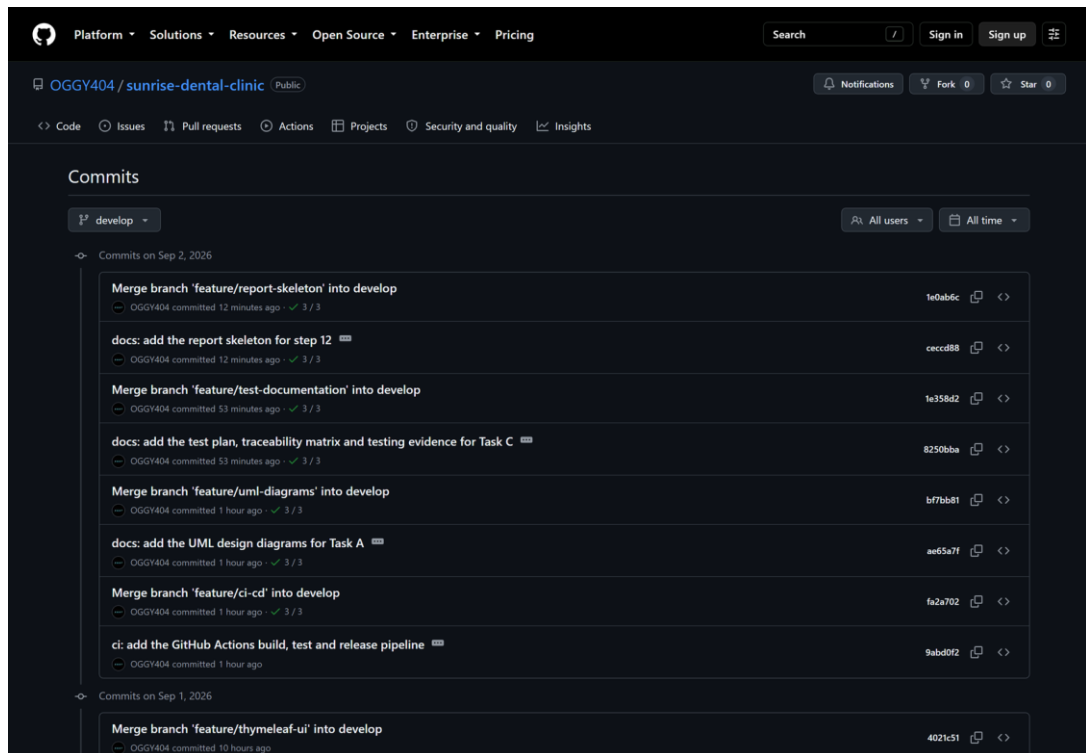


Figure 18: Commit history

Three-level branching approach was adopted:

- The main branch includes only the released versions and tags.
- The develop branch acts as an integration branch.
- The feature/* branch is divided into eleven branches based on the steps of the build process: database layer, UI, appointment reminders.

All feature branches were merged using --no-ff option; therefore, the history tree in Figure 18 displays the structure of the project, not a straight line.

5.2 Commits across multiple days

42 commits performed over four different days, with the implementation of one new feature per day: 6 on August 21st (repository, schema, procedures, triggers), 9 on August 28th (entities, patterns, services), 7 on September 1st (web services,

validation, security) and 20 on September 2nd (screens, reports, CI/CD, UML, documentation and reminders).

Each commit uses the standard format (feat:, fix:, test:, ci:, docs:) and provides a justification instead of simply repeating the diff content. Test: commits intentionally precede their respective feat: commits, and that's the TDD proof from §4.2.

There is also a .gitignore file to exclude target/, IDE files, and .env from version control since there has never been any credential committed (only .env.example is committed).

5.3 Continuous integration and deployment

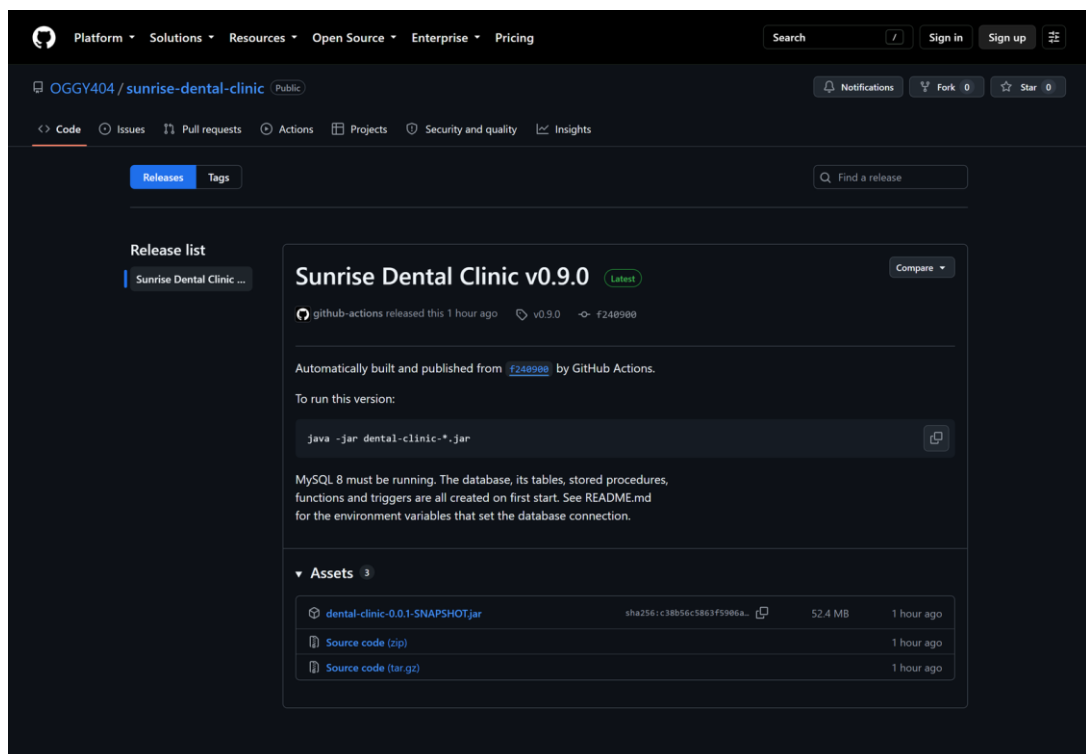


Figure 19: The release

ci.yml executes three jobs in sequence on each push/pull request: the 262 H2 tests, followed by the very same application using an actual MySQL 8 service container, and then the jar build, but only if the two former jobs pass successfully. The latter job is required due to the bugs mentioned in §4.6 – H2 corresponds only to the entity model it was created from, having no stored procedures, triggers, or generated columns.

release.yml is responsible for deployments. If a version tag is pushed, this will build from a fresh checkout and rerun the tests, as a tag could be pushed from any computer and a released version could never be an unverified version. It then releases the jar on a GitHub release.

v0.9.0 is published with the jar included (see Figure 19). Since Spring Boot embeds the web server in the jar file, the deployment requires only Java 17, MySQL 8 and one command:

```
export DB_HOST=localhost DB_NAME=sunrise_dental  
DB_USERNAME=clinic DB_PASSWORD=...  
export COOKIE_SECURE=true           # once served over HTTPS  
java -jar dental-clinic-0.0.1-SNAPSHOT.jar
```

The database, tables, functions and procedures will all be created automatically the first time the application starts, hence no setup is required for the new server.

6. Conclusion

The solution fulfills all six functional requirements plus 13 extra functionalities, validated by 268 automated tests, 8 UML diagrams and an automated pipeline that creates, tests and deploys a fully working product.

I am most proud that the double booking constraint is implemented within the database: no two receptionists could accidentally trigger a violation by saving at exactly the same time, and no Java code running outside the database could have provided such guarantee. What would have been changed is the constraint on treatment time, and starting the actual database testing much sooner.

The takeaway for me is more specific than “write tests”: a test proves nothing if it does not potentially disagree with your expectations.

References

- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley.
- Beck, K. (2002) *Test-Driven Development: By Example*. Boston, MA: Addison-Wesley.
- VMware (2024a) *Spring Framework Reference Documentation*. Available at: <https://docs.spring.io/spring-framework/reference/> (Accessed: 2 September 2026).
- VMware (2024b) *Spring Security Reference*. Available at: <https://docs.spring.io/spring-security/reference/> (Accessed: 2 September 2026).
- Oracle (2024) *MySQL 8.4 Reference Manual*. Available at: <https://dev.mysql.com/doc/refman/8.4/en/> (Accessed: 2 September 2026).
- OWASP (2024) *Cross-Site Request Forgery Prevention Cheat Sheet*. Available at: <https://cheatsheetseries.owasp.org/> (Accessed: 2 September 2026).

Appendices

Appendix	Contents	Source
A	Source code listings of the key classes	src/main/java/...
B	Full test plan, 111 cases	docs/testing/test-plan.md
C	Full traceability matrix	docs/testing/traceability-matrix.md
D	All 17 documented assumptions	docs/uml/README.md
E	Sequence diagram 4: cancel an appointment	docs/uml/png/08-sequence-cancel-appointment.png
F	Remaining screenshots	docs/report/screenshots/
G	End-to-end walkthrough output	docs/testing/manual-walkthrough-output.txt
H	Database scripts	src/main/resources/db/*.sql
I	Full list of the 17 web service endpoints	README.md